

CONTENTGEN

AI-Powered Blog Content Management System

PROJECT REPORT

A secure REST API platform for blog creation, content management, AI-assisted writing, and role-based publishing workflows.

Technology

Purpose

Node.js + Express.js

Backend runtime and REST API

MongoDB + Mongoose

Database and data modeling

Google Gemini AI

AI-assisted content generation and summarization

JWT + bcrypt.js

Authentication and password security

Postman / Thunder Client

API testing

Prepared as a structured academic project report

Introduction:

PROJECT TITLE:
AI BLOGGEST API

TEAM ID

6 ABA2ADF538 DA4003D63D

Team Leader:

Manikandan R
manikandanrengaraj18@gmail.com

Team Members:

Mohammed Jaffer A
ajjubhaljaffer444@gmail.com

Mohanakumar P
smileymohan12345@gmail.com

Michael E
emichael242006@gmail.com

Jayson A
iavsuiavson@gmail.com

1. Abstract

ContentGen is a modern backend platform designed to simplify blog creation and content management through a secure REST API. It combines traditional Content Management System features

with Generative AI so that users can create, organize, review, publish, and summarize blog content efficiently.

The system is implemented using Node.js and Express.js. MongoDB is used as the primary database, while Mongoose provides schema modeling and validation. Google Gemini AI is integrated for intelligent content generation and summarization. JWT authentication, role-based access control, bcrypt password hashing, request validation, sanitization, rate limiting, and structured logging improve security and reliability.

The architecture follows a modular layered approach. Routes receive requests, middleware verifies security rules, controllers coordinate business operations, services handle specialized logic, and Mongoose communicates with MongoDB. This separation makes the system easier to test, maintain, and extend.

The platform supports Admin, Editor, Author, and Reader roles. Each role receives a controlled set of permissions. Authors create content, Editors manage editorial queues, Administrators manage the complete environment, and Readers consume published content and participate through comments. Overall, ContentGen demonstrates how a conventional blog CMS can be enhanced with Generative AI while maintaining structured data, controlled permissions, and secure API operations.

Keywords: AI Content Generation, REST API, Node.js, Express.js, MongoDB, Mongoose, JWT, Gemini AI, CMS.

2. Introduction

Digital publishing has made blogging an important method for sharing information, knowledge, opinions, tutorials, and business content. However, managing a growing collection of blog posts can become difficult when authors must handle drafts, categories, tags, comments, publishing schedules, media information, and user permissions manually.

ContentGen addresses this challenge by providing a centralized backend API. Instead of connecting the frontend directly to the database, the frontend communicates with controlled REST endpoints. The backend validates requests, applies authorization rules, performs database operations, and returns structured JSON responses.

A major enhancement is the integration of Generative AI. A user can provide a topic or high-level instruction and request a complete blog draft. The AI service can also summarize long content into concise abstracts. These capabilities reduce repetitive writing effort while keeping the final content under the control of the user or editor.

The system is designed as a headless backend. Therefore, the View layer of the traditional MVC pattern is represented by API routes rather than server-rendered HTML pages.

2.1 Project Objectives

- Provide secure REST APIs for blog and user management.

- Support multiple user roles with controlled permissions.

- Store blogs, users, comments, categories, tags, and metadata reliably.

- Generate blog drafts using Google Gemini AI.

- Summarize long blog content using an AI service.

- Provide search and filtering capabilities.

- Support content approval, scheduling, and publishing workflows.

- Make the backend modular, maintainable, and scalable.

3. Scenario-Based Case Study

Background

Priya is a content creator who manages multiple blogs. As the number of posts grows, she finds it difficult to organize drafts, maintain categories and tags, schedule publications, moderate comments, and understand reader engagement.

Problem

Multiple drafts are difficult to organize and track.

Categories and tags can become inconsistent without a central taxonomy.

Post performance and engagement information is difficult to monitor.

Content approval and comment moderation consume time.

Media uploads and metadata require a structured registration process.

Different users need different permissions.

Solution

ContentGen provides a centralized backend API with authentication, role-based permissions, content workflows, category management, comment moderation, media metadata handling, and AI-assisted writing. The backend separates business logic from the frontend and applies a permission matrix to protected operations.

Usage

Author: creates posts, manages drafts, registers media pathways, and reviews permitted post metrics.

Editor/Admin: reviews content queues, changes publishing status, moderates comments, and manages permissions according to role.

Reader: views public posts and can submit comments through the permitted endpoint.

Outcome

By centralizing workflow controls, the system reduces content execution delays and maintains structural consistency. Schema validation protects database integrity, authorization prevents unauthorized administrative actions, and stored activity/analytics data supports later performance analysis.

4. Software and Hardware Requirements

4.1 Software Requirements

Software

Requirement / Purpose

Operating System

Windows 10/11, macOS, or Linux

Node.js

Version 16 or above; server runtime

npm

Version 8 or above; dependency management

Express.js

REST API and HTTP routing framework

MongoDB

NoSQL database for application data

Mongoose

ODM for schemas, validation, and database access

Postman / Thunder Client

API testing and verification

Visual Studio Code

Recommended development environment

4.2 Hardware Requirements

Component

Minimum / Recommended

Processor

Intel Core i5 8th Gen / AMD Ryzen 5 or equivalent

RAM

8 GB minimum; 16 GB recommended

Storage

At least 1 GB available project workspace

Network

Internet connection required for cloud AI services and package installation

4.3 Development Tools

VS Code is suitable for editing server files and running the integrated terminal. Postman or Thunder Client can be used to send HTTP requests and verify status codes, request validation, authentication, and JSON responses.

5. Project Architecture

ContentGen uses a layered server architecture. Each layer has a focused responsibility, which reduces coupling between API routing, security, business rules, database operations, and AI integrations.

Figure 1. High-level layered backend architecture.

Architecture Components

Interface: Postman, Thunder Client, or a frontend sends HTTP requests.

Express Server Gateway: listens on a port, maps HTTP methods, parses JSON, and handles CORS.

Authentication and Role Matrix: validates JWT tokens and checks permissions.

Services and Controllers: implement business rules for posts, comments, users, and AI operations.

Database Interface: Mongoose translates application operations into MongoDB queries.

AI Flow: communicates with the external Gemini AI service for generation and summarization.

6. Entity-Relationship Design

The database design uses MongoDB collections with logical references between entities. Although MongoDB is document-oriented, relationships are still defined through identifiers such as userID and post references.

Figure 2. Logical relationship model.

6.1 User Entity

Field

Type

Description

_id

ObjectId

Primary identifier

name

String

Required user name

email

String

Required and unique

password

String

Hashed password

role

String

Default: reader

6.2 Post Entity

Field

Type

Description

_id

ObjectId

Primary identifier

userID

ObjectId

Reference to author/user

name

String

Blog title

photo

String/Object

Image or media pathway metadata

message

String

Main blog content

likes

Number

Default: 0

timestamps

Boolean/Auto

Created and updated timestamps

7. MVC Architecture

The project follows the Model-View-Controller idea, adapted for a headless REST API.

Figure 3. MVC structure for the ContentGen backend.

Model Layer

The model layer contains Mongoose schemas and models. It defines field types, required values, defaults, validation rules, indexes, and relationships used when data is persisted in MongoDB.

Controller Layer

Controllers receive validated request data, invoke the required service or model operation, and format a consistent JSON response. Controllers should remain focused on request orchestration rather than containing every business rule.

View / API Layer

Because ContentGen is a backend-only REST service, there is no traditional HTML view engine. API routes act as the interface through which clients access controller functionality.

Benefits

- Separation of responsibilities.
- Easier debugging and unit testing.
- Reusable business logic.
- Simpler maintenance as features increase.
- Better support for multiple frontend clients.

8. User Roles and Responsibilities

Role

Main Responsibilities

Admin

Manage users, roles, comments, system settings, logs, and administrative operations.

Editor

Review drafts, manage editorial queues, publish approved content, and maintain categories.

Author

Create and manage own blog content within assigned permissions.

Reader

Read published posts and submit comments through allowed routes.

8.1 Role-Based Access Control

Role-based access control ensures that an authenticated user is not automatically allowed to perform every operation. A route can require authentication and then check whether the user's role is included in the permitted role list.

Example policy concept:

Operation

Admin

Editor

Author

Reader

Manage users

Yes

Limited

No

No

Create own post

Yes

Yes

Yes

No

Review pending posts

Yes

Yes

Limited

No

Read published posts

Yes

Yes

Yes

Yes

Moderate comments

Yes

Yes

No

No

The exact permissions should be implemented consistently in middleware and route definitions rather than being trusted from client-side input.

9. Security Architecture

Security is implemented as multiple layers rather than relying on a single mechanism.

Figure 4. Request security pipeline.

JWT Authentication

After successful login, the server issues a signed JSON Web Token. Protected requests include the token in the Authorization header. Middleware verifies the signature and extracts the authenticated user context.

Password Hashing

Passwords should never be stored as plain text. bcrypt.js can generate a secure hash during registration and compare a login password against the stored hash.

Validation and Sanitization

Request validation checks data types, required fields, lengths, and allowed values. Sanitization reduces the risk of unsafe payloads, including common cross-site scripting and NoSQL injection patterns.

Rate Limiting and Logging

Rate limiting reduces excessive repeated requests. Structured logs help developers identify failures, authentication problems, and important operational events without exposing sensitive credentials.

10. Blog Management Module

The blog module provides the central CRUD functionality of the system. CRUD means Create, Read, Update, and Delete.

Operation

Purpose

Create

Create a new blog draft or permitted post.

Read

Fetch one post or a list of posts.

Update

Edit title, content, tags, status, media metadata, or other permitted fields.

Delete

Remove a post when the authenticated role has permission.

Figure 5. Suggested blog lifecycle.

Publishing States

Draft: content is being prepared.

Pending Approval: content is waiting for editorial review.

Scheduled: approved content has a planned publication time.

Published: content is publicly available.

11. AI Content Generation

The AI module is the main intelligent feature of ContentGen. It accepts a topic or prompt, sends controlled context to the configured Gemini AI service, and returns generated content to the application.

Figure 6. AI-powered blog generation flow.

Typical Endpoint

POST /api/ai/generate-blog

The endpoint can accept structured input such as a topic, desired tone, target audience, approximate length, and optional keywords. The controller should validate these values before invoking the AI service.

Example request concept

A client might submit a JSON body containing a topic such as "Benefits of Cloud Computing", a target audience, and a preferred writing style. The AI service returns a structured draft containing a title, introduction, sections, and conclusion.

Important Design Principle

AI output should be treated as generated assistance rather than automatically trusted database content. Validation, moderation, and human review can be applied before publication.

12. AI Summarization and Intelligent Features

ContentGen can provide an additional AI endpoint for summarizing long blog content.

Typical Endpoint

POST /api/ai/summarize

The endpoint receives article content and asks the AI service to produce a concise summary. The summary can be used for previews, metadata, search descriptions, or editorial review.

AI FAQ Assistance

An FAQ feature can use the same AI integration pattern to generate question-and-answer suggestions from a blog topic or article. The generated FAQs should be reviewed before being presented as authoritative information.

WeatherWise and FitTrack

The supplied project concept also identifies AI WeatherWise insights and AI FitTrack recommendations as intelligent services. These should be implemented as separate service modules so that their external data dependencies and validation rules remain isolated from the core blog module.

Benefits of Service Separation

- AI provider changes can be made in one service layer.

- Controllers remain small and readable.

- Different prompts can be versioned independently.

- Failures from external AI services can be handled centrally.

- Testing becomes easier because AI calls can be mocked.

13. API Design and Testing

Figure 7. Typical API testing sequence.

Core API Categories

Category

Example Endpoint

Purpose

Authentication

/api/auth/register

Create user account

Authentication

/api/auth/login

Authenticate user and issue JWT

Blogs

/api/posts

Create or fetch posts

AI

/api/ai/generate-blog

Generate blog draft

AI

/api/ai/summarize

Summarize article

Comments

/api/comments

Create and manage comments

Search

/api/posts/search

Find content by supported criteria

Postman Testing Procedure

Start the backend server.

Register a test user using the registration endpoint.

Login and copy the returned JWT.

Add the token to the Authorization header for protected requests.

Create a test blog and verify the JSON response.

Fetch the blog list and verify stored data.

Test AI generation and summarization endpoints.

Test invalid input and unauthorized access to confirm security behavior.

14. Database Configuration

MongoDB acts as the persistent data store. Mongoose provides the schema and model layer used by the Node.js application.

Recommended Connection Process

Install the mongoose package.

Store the MongoDB connection string in an environment variable.

Create a dedicated database connection module.

Connect to MongoDB when the application starts.

Handle connection failures and prevent dependent operations from running before the connection is ready.

Define schemas and models for users, posts, comments, categories, and related resources.

Example installation command

```
npm install mongoose
```

Project data should be validated at the schema level and, where appropriate, again at the request level. Unique indexes such as email uniqueness should be handled carefully because database constraints provide stronger guarantees than client-side checks alone.

Environment Configuration

Variable

Example Purpose

PORT

HTTP server port

MONGODB_URI

MongoDB connection string

JWT_SECRET

Secret used to sign and verify tokens

GEMINI_API_KEY

Credential for the configured AI service

Sensitive values such as database passwords, JWT secrets, and AI credentials should not be hard-coded into source files or committed to public repositories.

15. Project Setup and Configuration

The project can be created as a Node.js backend with a dedicated Server directory.

Step 1 – Create Project Folder

Create a new folder using the project name.

Create a Server folder inside it.

Open the Server folder in Visual Studio Code.

Step 2 – Initialize npm

`npm init -y`

Step 3 – Create Core Files and Folders

Item

Responsibility

`server.js`

Application entry point / server startup

`app.js`

Express application configuration

`models/`

Mongoose schemas and models

`controllers/`

Request handling and business orchestration

`routes/`

API endpoint definitions

`middleware/`

Authentication, authorization, validation, and error handling

`services/`

AI and other specialized business services

`config/`

Database and application configuration

Suggested Architecture

A modular structure helps keep the project readable as the number of endpoints grows. Each feature can contain its own route, controller, service, and model logic where appropriate.

Server Startup

The application should load environment configuration, establish the database connection, initialize Express middleware and routes, and finally start listening on the configured port.

16. Error Handling and Data Integrity

A centralized error-handling mechanism provides consistent API responses. Instead of returning different error formats from every controller, the application can pass errors to one middleware layer.

Common Error Categories

Status

Meaning / Example

400

Invalid request body or validation failure

401

Missing or invalid authentication token

403

Authenticated user lacks required role

404

Requested resource does not exist

409

Conflict such as duplicate unique value

429

Rate limit exceeded

500

Unexpected server-side failure

Data Integrity Measures

Required fields are validated before persistence.

Mongoose schemas enforce data types and defaults.

Unique indexes reduce duplicate identity records.

Authorization rules protect ownership-sensitive operations.

API output is validated before it is treated as publishable content.

Database errors are handled without exposing internal secrets.

Centralized Error Response

A useful JSON error response should contain a clear message, an appropriate HTTP status, and optionally a safe error code. Internal stack traces and credentials should not be returned to clients in production.

17. Key Features and Advantages

Key Features

- JWT-based stateless authentication.
- Role-based access control for Admin, Editor, Author, and Reader.
- Secure bcrypt password hashing.
- CRUD operations for blog content.
- Categories and tags for content organization.
- Comment creation and moderation workflow.
- Media metadata/path registration.
- Advanced search and filtering.
- AI blog generation and summarization.
- Request validation and sanitization.
- Rate limiting and structured logging.
- Modular architecture for future extensions.

Advantages

- Reduces repetitive content creation work.
- Centralizes blog management in one backend.
- Improves security through layered controls.
- Supports multiple user types and workflows.
- Can serve different frontend applications through the same API.
- Scales more easily because domain logic is separated into modules.
- AI services can be extended without rewriting the core blog engine.

Limitations and Considerations

- AI-generated content requires human review for accuracy and relevance.
- External AI services may have usage limits, latency, or cost.
- Production deployment requires stronger secrets management and monitoring.
- Analytics are only meaningful when events are consistently captured.

18. Testing, Deployment and Future Enhancements

Testing Strategy

- Unit testing for validation and service functions.
- Integration testing for routes and database interactions.
- Authentication tests for valid, expired, and invalid JWTs.
- Authorization tests for each role.
- Negative testing for malformed input and missing fields.
- AI endpoint tests using controlled sample prompts and mocked failures.
- API regression testing through Postman collections.

Deployment Considerations

For production use, the API can be deployed on a cloud server or container platform. MongoDB can be hosted using a managed service. Environment variables should be configured through the deployment platform, and HTTPS should be enabled so credentials and tokens are protected in transit.

Future Enhancements

- Rich analytics dashboard for views, likes, comments, and engagement.
- Scheduled publishing with background job processing.
- Version history and rollback for blog posts.
- Image upload service with automatic optimization.
- Full-text search using a dedicated search engine.
- AI-assisted SEO title, keyword, and metadata generation.
- Plagiarism/originality checking integration.
- Notification system for approvals and comments.
- Audit trails for administrative actions.
- Containerized deployment with CI/CD automation.

19. Conclusion

ContentGen is a modern AI-powered backend solution for centralized blog creation and content management. The project combines Node.js, Express.js, MongoDB, Mongoose, JWT authentication, bcrypt password hashing, role-based authorization, and Google Gemini AI into a modular REST API architecture.

The system addresses practical problems faced by content creators such as managing drafts, organizing categories and tags, moderating comments, controlling publication workflows, and producing content efficiently. The AI integration adds intelligent generation and summarization capabilities while the security architecture ensures that sensitive operations remain protected.

The layered design also makes the application easier to maintain and extend. Routes, controllers, services, models, middleware, and external integrations can evolve independently. With appropriate testing, monitoring, human review of AI output, and secure deployment practices, the platform can serve as a strong foundation for a scalable content management application.

20. References / Technology Stack

Technology

Role in Project

Node.js

JavaScript server runtime

Express.js

REST API framework

MongoDB

NoSQL database

Mongoose

MongoDB ODM and schema validation

Google Gemini AI

Generative AI integration

JWT

Stateless authentication

bcrypt.js

Password hashing

Postman / Thunder Client

API testing

Visual Studio Code

Development environment

End of Project Report